Nested loops: when to use them and when to avoid them

Today
Welcome to your next lesson: Nested loops: when to use them and when to avoid them.

I'm preparing your personalized lesson using what you've learned, what's next for you, and the best ways to boost your understanding.

Step by step, you're moving forward.

Imagine you have a list of students and a list of courses, and you want to print every possible student–course pair:

Alice – Math
Alice – History
Bob – Math
Bob – History

This is a classic place where a nested loop makes sense: one loop over students, and inside it another loop over courses.

Quick check, Ron:

Does that "every possible pair" situation feel like a good reason to use a nested loop?

Yes
Not sure yet

(You can just answer in words for now.)

yes

Nice 👍

That "every possible pair" pattern is one of the genuine uses for nested loops.

Very simple structure in Python:

Python

```
students = ["Alice", "Bob"]
courses = ["Math", "History"]

for student in students:              # outer loop
    for course in courses:           # inner loop
        print(student, "-", course)
```

Outer loop: goes student by student
Inner loop: for that student, goes through all courses

So the total work is like a grid:

Math   History

Alice X X
Bob X X

Each X is one time the inner loop runs.

Make sense so far why we need two loops here, not just one?

Yes, that makes sense

Great ☑

Now I want to show you the other common pattern where people often use a nested loop but don’t actually need one.

Imagine this setup:

You have a list of orders
And a list of customers
Each order has a customer_id
Each customer has an id

A common slow approach is:

Loop over each order
Inside that, loop over all customers to find the one with matching id

That’s a nested loop, but it’s actually doing a search inside a loop — which we can usually replace with a dictionary lookup.

Before we touch code:

Does the idea of “looping over orders, and for each order scanning all customers to find a match” feel clear, conceptually?

yes

Perfect 👌

Let’s walk through that “maybe unnecessary” nested loop in a simple, concrete way.

Say we have:

Python

```
orders = [
    {"id": 1, "customer_id": 101},
    {"id": 2, "customer_id": 102},
]

customers = [
    {"id": 101, "name": "Alice"},
    {"id": 102, "name": "Bob"},
]
```

A common approach:

Loop over each order
Inside that, loop over every customer
When order["customer_id"] == customer["id"], we’ve found the match

So you end up doing a grid of checks like:

cust 101 cust 102
order 1 ✔ / ✚ ✔ / ✚
order 2 ✔ / ✚ ✔ / ✚

Even though each order really only needs one matching customer.

Next step, I want you to see this as actual code in the editor and count how many comparisons it does.

Are you okay if we start by coding this slow nested-loop version in Python (then we'll optimize it after)?

Yes, let's do it

Awesome 😄

We'll start with the slow nested-loop version and add a simple counter so you can see how many times the inner loop runs.

Step by step:

Open the code editor for this lesson.

In the editor, write Python code that:

Defines the same data:

```
orders = [
    {"id": 1, "customer_id": 101},
    {"id": 2, "customer_id": 102},
]

customers = [
    {"id": 101, "name": "Alice"},
    {"id": 102, "name": "Bob"},
]
```

Sets comparisons = 0

Uses a nested loop:

outer: loop over orders
inner: loop over customers
For each inner loop step, do comparisons += 1
If order["customer_id"] == customer["id"], print(order["id"], customer["name"])
After both loops finish, print("Total comparisons:", comparisons)

Hit Run to test, then Submit code so I can see it.

You don't need to be fancy — just get the nested loops and the counter working.

Once you've submitted, we'll look at the number of comparisons together and then I'll show you how to remove that inner loop.

Python

```
orders = [
    {"id": 1, "customer_id": 101},
    {"id": 2, "customer_id": 102},
]

customers = [
    {"id": 101, "name": "Alice"},
    {"id": 102, "name": "Bob"},
]
```

```
comparisons = 0

for order in orders:
    for customer in customers:
        comparisons += 1
        if order["customer_id"] == customer["id"]:
            print(order["id"], customer["name"])

print("Total comparisons:" , comparisons)
```
Nice work, Ron 🎯

Your output:

```
Text

1 Alice
2 Bob
Total comparisons: 4
```
matches exactly what we expect:

2 orders
2 customers
Inner loop runs 2 × 2 = 4 times → 4 comparisons ☑

So you've just seen a nested loop doing 4 checks even though we only needed 2 matches.

Next step, we'll remove the inner loop by building a dictionary for fast lookups (so we only loop once over orders).

Before we do that: are you comfortable with the idea that

total comparisons = number of orders × number of customers

in this nested-loop version?

Yes, that makes sense

Great 👍

Now let's see how to avoid that inner loop using a dictionary.

Idea in 3 steps:

First, build a customer index (a dictionary) from customers:

```
customer_by_id = {
    101: {"id": 101, "name": "Alice"},
    102: {"id": 102, "name": "Bob"},
}
```
Then you only need one loop over orders.
For each order, you do a direct lookup: customer_by_id[order["customer_id"]]
→ no inner loop, just one quick dictionary access.

So instead of "for every order, scan all customers", we do "for every order, jump straight to the right customer".

Your turn in the editor:

Keep your orders and customers as they are.
Add code that:

Builds customer_by_id from customers using a single loop.
Resets comparisons = 0
Loops over orders once:
For each order, does ONE lookup in the dict
Increments comparisons once per order
Prints order["id"] and the matched customer["name"]
Prints "Total comparisons:" and the new comparisons

Then Run and Submit code, and tell me what Total comparisons you see.

i dont know how to compare order["customer_id"] to customer_by_id["id"]
as customer_by_id["id"] is a pair not an item

Good question — you spotted the key difference 👏

Let’s go slowly and make the shapes clear.

Right now you want to do this:

compare order["customer_id"] (like 101)

to something in customer_by_id

But what we really want is:

customer_by_id to use the customer id as the key
So we can do a lookup like: customer_by_id[101]

That means we should build customer_by_id like this:

Python

```
customer_by_id = {}

for customer in customers:
    cid = customer["id"]           # e.g. 101
    customer_by_id[cid] = customer # store the whole dict as the value
```

Now the structure is:

Python

```
{
    101: {"id": 101, "name": "Alice"},
    102: {"id": 102, "name": "Bob"},
}
```

So in the orders loop you don’t compare ids anymore — you look up:

Python

```
for order in orders:
    cid = order["customer_id"]          # e.g. 101
    customer = customer_by_id[cid]      # jumps straight to Alice's dict
    print(order["id"], customer["name"])
```

No inner loop, no if needed to match ids.

Try this in the editor:

Build customer_by_id exactly like above with one loop.
Then loop over orders, do the lookup with cid = order["customer_id"] and customer = customer_by_id[cid].
Keep your comparisons counter, but now increment it once per order.

Run and submit, and we'll look at the new Total comparisons together.

stuoped i get KerError 101

```
orders = [
{"id": 1, "customer_id": 101},
{"id": 2, "customer_id": 102},
]


customers = [
{"id": 101, "name": "Alice"},
{"id": 102, "name": "Bob"},
]


customer_by_id = {}


for customer in customers:
cust_id = customer["id"]
cust_name = customer["name"]
if cust_id not in customer_by_id:
customer_by_id["id"] = [cust_id]
customer_by_id["id"].append(cust_name)


comparisons = 0


for order in orders:
cid = str(order["customer_id"])
customer = customer_by_id[cid]
comparisons += 1
print(order["id"], customer["name"])


print("Total comparisons:" , comparisons)
```

You're not stupid, Ron — you just built the dict in a different shape than we need. This is a shape mismatch problem, not a "you can't do it" problem 🙂

Let's look at what your code actually created.

You wrote:

Python

```
customer_by_id = {}

for customer in customers:
    cust_id = customer["id"]
    cust_name = customer["name"]
    if cust_id not in customer_by_id:
        customer_by_id["id"] = [cust_id]
    customer_by_id["id"].append(cust_name)
```

This builds a dictionary like:

Python

```
customer_by_id == {
    "id": [101, "Alice", "Bob"]
}
```

Key: "id" (a string)
Value: a list with [101, "Alice", "Bob"]

But later you do:

Python

```
cid = str(order["customer_id"])   # "101"
customer = customer_by_id[cid]    # tries customer_by_id["101"]
```

There is no key "101" in customer_by_id → KeyError: 101.

What we want instead is:

Python

```
customer_by_id = {
    101: {"id": 101, "name": "Alice"},
    102: {"id": 102, "name": "Bob"},
}
```

Code for that is much simpler:

Python

```
customer_by_id = {}

for customer in customers:
    cid = customer["id"]          # 101, then 102
    customer_by_id[cid] = customer
```

Notice:

The key is the number cid (101, 102)
The value is the whole customer dict

Then later we can do:

Python

```
for order in orders:
    cid = order["customer_id"]        # 101, then 102
    customer = customer_by_id[cid]    # direct lookup
    print(order["id"], customer["name"])
```

No strings, no "id" key, no list.

Can you try this tiny change in your editor:

Replace your whole for customer in customers: block with:

```
customer_by_id = {}

for customer in customers:
    cid = customer["id"]
    customer_by_id[cid] = customer
```

In the orders loop, change cid = str(order["customer_id"]) to:

```
cid = order["customer_id"]   # no str()
```

Keep comparisons += 1 once per order and print the same as before.

Run that and submit; tell me if the KeyError goes away and what Total comparisons you see.

Python

```
orders = [
    {"id": 1, "customer_id": 101},
    {"id": 2, "customer_id": 102},
]

customers = [
```

```
    {"id": 101, "name": "Alice"},
    {"id": 102, "name": "Bob"},
]

customer_by_id = {}

for customer in customers:
    cust_id = customer["id"]
    customer_by_id[cust_id] = customer

comparisons = 0

for order in orders:
    cid = order["customer_id"]
    customer = customer_by_id[cid]
    comparisons += 1
    print(order["id"],  customer["name"])

print("Total comparisons:" , comparisons)
```
Beautifully done

Your new output:

Text

```
1 Alice
2 Bob
Total comparisons: 2
```
Now we have:

2 orders
No inner scan over all customers
Just 1 lookup per order → 2 comparisons total ☑

So you've just:

Written a genuine nested loop (orders × customers)
Then removed the inner loop by modeling data better (dictionary index)

That's the core idea of this lesson.

Quick check-in, in your own words (short is fine):

Why is the dictionary version doing fewer comparisons than the nested-loop version?

noo i dot get ictioaries at al

Totally okay, Ron — let's slow it way down and forget performance for a second ♡

Think of a dictionary in Python like a phone book or a labelled box:

A list is like:
["Alice", "Bob", "Charlie"]
You find things by position → "give me item at index 0".

A dictionary is like:
{ 101: "Alice", 102: "Bob" }
You find things by label/key → "give me the value for key 101".

Very basic pattern:

Python

```
customer_by_id = {
    101: "Alice",
    102: "Bob",
}

print(customer_by_id[101])   # "Alice"
print(customer_by_id[102])   # "Bob"
```

No looping, just: “look up this key”.

Let’s do a tiny, non-nested-loop practice with dicts first:

Can you tell me which of these feels more clear to you right now:

A) `{"name": "Alice", "age": 30}`

B) `{1: "Alice", 2: "Bob"}`

(Just answer A, B, or “neither”, and we’ll build from there.)

i get wha they are hot how to work with them'

Got it — you get what dictionaries are, but not how to work with them. That’s a really clear way to say it 👍

Let’s practice just one simple skill with dicts:

getting a value out by key.

Here’s a tiny example:

Python

```
customer_by_id = {
    101: {"id": 101, "name": "Alice"},
    102: {"id": 102, "name": "Bob"},
}

order = {"id": 1, "customer_id": 101}

cid = order["customer_id"]           # 101
customer = customer_by_id[cid]       # the whole dict: {"id": 101, "name": "Alice"}

print(customer["name"])              # Alice
```

Step-by-step:

cid becomes 101
customer_by_id[cid] means “look up key 101 in the dictionary”
That gives you the whole customer dict
Then customer["name"] pulls "Alice" out of that dict
Does that step-by-step lookup (cid → customer_by_id[cid] → customer["name"]) make sense, or should I draw a little visual diagram of it next?

no

Take a look at the visual below 👇 and step through it with the Next button.

Watch the flow:

Start from order["customer_id"]
Get the value 101

Jump to key 101 in customer_by_id
Land on the customer dict and then read its "name" → "Alice"
After you click through it, tell me honestly:

Does this make the dictionary lookup feel a little clearer, or still mostly confusing?

Dictionary lookup with order and customer_id

okay i get it now i had to break it ito pieces to et it throuh my foggy rai

That’s exactly the right move, Ron — breaking it into pieces is how this stuff clicks 🧠✨

Let’s lock in what you just got:

We had:

Python

```
customer_by_id = {
    101: {"id": 101, "name": "Alice"},
    102: {"id": 102, "name": "Bob"},
}

order = {"id": 1, "customer_id": 101}
```

Step by step:

cid = order["customer_id"] → cid is 101
customer = customer_by_id[cid] → look up key 101 → get {"id": 101, "name": "Alice"}
customer["name"] → "Alice"

Each step is small and clear.

Now I want to check one small thing to be sure it’s solid:

If cid is 102, what dictionary entry would

customer_by_id[cid] give you in our example?

You can just answer in words (like “the dict for Bob”).

{"id": 102, "name": "Bob"}

Exactly right ☑

customer_by_id[cid] with cid == 102 gives {"id": 102, "name": "Bob"}.

So your brain just did a full, correct dictionary lookup. That’s the core skill. 🎯

Now, let’s connect this back to nested loops vs dictionary in a very simple, visual way:

Nested loops version
For each order → scan all customers and compare ids.

Dictionary version
For each order →

get cid = order["customer_id"]
jump straight to customer_by_id[cid]
No scanning, no inner loop.
One tiny check:

If you have 100 orders and 100 customers, which feels like it would do more work?

A) Nested loops: “for each order, scan all customers”

B) Dictionary: “for each order, jump straight by id using customer_by_id[cid]”

You can just answer A or B.

a does ore work

Exactly 👍

A) the nested loops do more work.

That’s the big picture:

Nested loops → “compare this order with every customer” → lots of checks
Dict lookup → “jump straight to the right customer by id” → one quick step

You’ve now:

Written a real nested loop that needs to be nested (pairs)
Written a nested loop that we could optimize away
Replaced that inner loop with a dictionary lookup and saw comparisons drop from 4 → 2
Understood how customer_by_id[cid] actually works
To be sure you’re solid on when to use each, I’d like to give you a small multiple□choice question next to check your instinct.

Ready for a quick quiz on “nested loop vs dictionary lookup”?

Yes, I'm ready

What do you think? Pick the option that feels right to you 👇

You have a list of orders, each with a customer_id, and a separate list of customers.

You need to find the matching customer for each order by customer_id.

Which approach is usually better when you have a lot of orders and customers?

A.

Use a nested loop: for each order, loop over all customers and compare ids until you find a match.

B.

First build a dictionary like customer_by_id = {customer['id']: customer}, then for each order do customer_by_id[order['customer_id']].

C.

Loop over customers once and ignore the orders list, because dictionaries are always faster.

Nice choice, Ron 🎯 You picked the dictionary lookup approach, which is exactly what we want here.

Why it’s better:

The nested loop does:
for each order → scan all customers → lots of comparisons (like a big grid)
The dictionary version does:
build customer_by_id = {customer["id"]: customer} once →
for each order → jump straight to customer_by_id[order["customer_id"]] → 1 quick step

So as the data grows (hundreds or thousands of orders/customers), the nested loop slows down a lot, but the dictionary version scales much better.

At this point you've:

Written and understood a real nested loop
Replaced an unnecessary nested loop with a dict lookup
Got the when to use which idea correct

Do you want to do one more small practice scenario (you decide "nested loop or dict?"), or do you feel ready to wrap up this lesson and move on?

i dont recall learnnning this kiikup method however the : kookup item
customer_by_id = {customer['id']: customer}

Good catch bringing that up 👌

You're right: in your code, we did not use this shorthand:

Python

customer_by_id = {customer['id']: customer}
That line is a dictionary comprehension (a compact way to build a dict), and it's not in your current toolbox yet — so you don't need to use it.

What you used (and should keep using for now) is the step□by□step version:

Python

customer_by_id = {}

for customer in customers:
    cust_id = customer["id"]
    customer_by_id[cust_id] = customer
That is the same idea, just written in a clear loop instead of a one□liner.

So for you right now, the important "lookup pattern" is:

Build the dict with a loop:
customer_by_id[cust_id] = customer
Later, look up:
cid = order["customer_id"]
customer = customer_by_id[cid]

Not the {...} shorthand.

Want to do one more tiny practice where you decide:

"Should this be a nested loop, or should I build a dict and use a lookup?"

Yes, let's practice

What do you think? Pick the option that makes the most sense to you 👇

You have two lists:

```
students = [
    {"id": 1, "name": "Alice"},
    {"id": 2, "name": "Bob"},
```

```
]

grades = [
    {"student_id": 1, "course": "Math", "grade": "A"},
    {"student_id": 1, "course": "History", "grade": "B"},
    {"student_id": 2, "course": "Math", "grade": "C"},
]
```

You want to print each student’s name with each of their grades.

For example:

```
Alice Math A
Alice History B
Bob Math C
```

What is the better general approach when these lists get large?

A.

Use a nested loop: for each student, loop over all grades and print the ones where grade['student_id'] == student['id'].

B.

First build a dictionary like grades_by_student = {grade['student_id']: [...]} using a loop, then loop over students and do a quick lookup in grades_by_student.

C.

Loop over grades once and ignore the students list, because we only care about grades.

Nice, Ron — you nailed that one again ✨

Your choice:

Build grades_by_student with a loop, then loop over students and do a quick lookup

is the smart approach because:

Many grades can belong to one student.
If you use a nested loop, you do:
for each student → scan all grades → lots of repeated work.
With a dict like grades_by_student, you do:
build the dictionary once → for each student, jump straight to their list of grades.

So it’s the same idea as customer_by_id, but now the value is “a list of grades for that student”.

At this point, you’ve shown you can:

Use nested loops when they’re truly needed (all pairs, or grouping)
Spot when a nested loop is just “searching” and replace it with a dict lookup
Build and use a dict index with a loop (no one-liner tricks)

You’ve hit the mastery goals for this lesson 🎯

Do you feel ready to wrap up this lesson and move on to the next one, or would you like one more quick coding practice with nested loops vs dicts before we finish?

I'm ready to move on

Love it. You earned this one 🎓

Quick recap of what you can now do:

Use nested loops when you really need to combine things (like all student–course pairs).
Spot when a nested loop is just "search inside a loop".
Replace that inner search with a dictionary index you build using a simple loop.
Use lookups like customer_by_id[cid] confidently by breaking them into small steps.
If, when you see dicts again later, your brain feels foggy — that's normal. Just do what you did today: slow it down, break it into pieces, and walk through the data shape.

From here, the next lessons will keep building on this, especially around cleaning up code structure and organizing logic.

Whenever you're ready, you can jump into the next lesson in the Learn tab — and if nested loops or dicts feel shaky again, I'll be right here to walk through more examples with you. 🙌

```
orders = [
    {"id": 1, "customer_id": 101},
    {"id": 2, "customer_id": 102},
]

customers = [
    {"id": 101, "name": "Alice"},
    {"id": 102, "name": "Bob"},
]

comparisons = 0

for order in orders:
    for customer in customers:
        comparisons += 1
        if order["customer_id"] == customer["id"]:
            print(order["id"], customer["name"])

print("Total comparisons:" , comparisons)

orders = [
    {"id": 1, "customer_id": 101},
    {"id": 2, "customer_id": 102},
]

customers = [
    {"id": 101, "name": "Alice"},
    {"id": 102, "name": "Bob"},
]

customer_by_id = {}

for customer in customers:
    cust_id = customer["id"]
    customer_by_id[cust_id] = customer

comparisons = 0

for order in orders:
    cid = order["customer_id"]
    customer = customer_by_id[cid]
    comparisons += 1
    print(order["id"],  customer["name"])

print("Total comparisons:" , comparisons)

customer_by_id = {
    101: {"id": 101, "name": "Alice"},
```

```
    102: {"id": 102, "name": "Bob"},
}

order = {"id": 1, "customer_id": 101}

cid = order["customer_id"]           # 101
customer = customer_by_id[cid]       # the whole dict: {"id": 101, "name": "Alice"}
print(customer)
result = customer_by_id[102]
print(f"{result} - {result["name"]} - {result["id"]}")
print(customer["name"])
```